Final Project Report: Logic-Based Encrypted Inference

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Theoretical Background of Secure Computing

The fundamental question of modern cryptography is no longer 'How do we hide a message?', but rather 'How do we manipulate a hidden message without revealing it?'. Standard encryption (AES/RSA) transforms data into a static, locked state. To perform any robust computation?whether it be a financial audit, a voting tally, or a neural network prediction?this 'lock' must be broken, and the data exposed to the processor's memory. This architectural flaw means that 'Cloud Computing' and 'Total Privacy' have historically been mutually exclusive concepts. Homomorphic Encryption (HE) fundamentally resolves this paradox by enabling algebraic operations (Addition and Multiplication) to be performed directly on the ciphertext space.

1.2 The "Data-in-Use" Vulnerability

In a typical Client-Server architecture, data exists in three states: At Rest, In Transit, and In Use. While strict standards like TLS 1.3 protect data in transit, and AES-256 protects data at rest, data 'In Use' remains the weak link. When a Neural Network performs inference, it requires the raw floating-point numbers to perform matrix multiplication. This project implements a solution where the Neural Network itself operates blindly, accepting encrypted inputs and producing encrypted outputs, ensuring the data remains mathematically opaque even to the machine processing it.

CHAPTER 2: LITERATURE REVIEW

2.3 The CKKS Scheme vs BGV/BFV

For Privacy-Preserving Machine Learning (PPML), the choice of encryption scheme is critical. Early generations like BGV (Brakerski-Gentry-Vaikuntanathan) and BFV (Brakerski/Fan-Vercauteren) were designed for exact integer arithmetic. While perfect for voting systems, they fail for Neural Networks, which rely on weighted sums of decimals (e.g., 0.334 * 1.5). The Cheon-Kim-Kim-Song (CKKS, 2017) scheme introduced the concept of 'Approximate Homomorphic Encryption'. It treats messages as vectors of complex numbers and allows for a small, manageable amount of error (noise) in the decryption, much like standard floating-point definition. This project specifically leverages CKKS because its structure maps 1:1 with the Linear Algebra operations required by Multi-Layer Perceptrons (MLPs).

CHAPTER 4: CRYPTOGRAPHIC THEORY

4.1 Ring Learning With Errors (RLWE)

The security of our system relies on the Hardness Assumption of the Ring Learning With Errors (RLWE) problem. Unlike RSA, which relies on factoring large primes (which Quantum computers could theoretically break via Shor's Algorithm), RLWE is considered 'Post-Quantum Secure'. It involves finding the secret value 's' given a list of polynomial equations with added noise terms 'e'. Because the operations occur over a polynomial ring, the computational complexity of breaking the key scales exponentially with the dimension of the lattice.

4.2 Evaluation Keys (Relinearization)

A unique challenge in HE is that multiplying two ciphertexts expands the size of the ciphertext polynomial (from degree N to degree N^2). If left unchecked, a few multiplications would make the data too large for any computer to handle. To solve this, our system implements 'Relinearization'. The client generates a special public key called the 'Evaluation Key' (or Relinearization Key). The Server uses this key to compress the ciphertext back to its original size (degree N) after every multiplication layer, keeping the implementation efficient.

CHAPTER 5: IMPLEMENTATION DETAILS

5.1 Key Generation & Context Management

The system context is defined by three parameters: The Polynomial Modulus Degree (N), the Coefficient Modulus (Q), and the Scale (S). For our implementation, we selected N=2^14 (16384). This high degree is necessary to support the 'depth' of multiplications required by a Neural Network without running out of noise budget. We utilize the Pyfhel library to manage these contexts, ensuring that the Public Key, Secret Key, and RelinKey are generated consistently on the client side.

5.3 Handling Non-Linear Activations

Homomorphic Encryption only supports Addition and Multiplication. It cannot compute 'Sigmoid(x) = 1/(1+e^-x)' because division and exponentiation are not polynomial operations. Previous works approximated Sigmoid using Taylor Series polynomials (e.g., x - x^3/6...), but this introduces massive approximation errors. Our implementation bypasses this theoretical limit via a 'Split-Computation Protocol'. Instead of forcing the server to compute the Sigmoid, the server sends the encrypted linear result (W*x+b) back to the client. The client (who has the key) decrypts it, applies the perfect Sigmoid function, re-encrypts it, and returns it. This ensures 100% mathematical accuracy while keeping the server blind to the actual values.

CHAPTER 6: CASE STUDY & VALIDATION

To validate this General Purpose Crypto-Architecture, we applied it to two specific medical classification tasks. Note: These are merely applications of the core engine; the engine itself is agnostic to the data type.

6.1 Case Study 1: Diabetes Prediction (8-Vector Input)
- The input consists of an encrypted vector of size 8.
- The system successfully performed the encrypted dot product against a [8x12] weight matrix.

6.2 Case Study 2: Heart Disease Prediction (13-Vector Input)
- Demonstrates scalability to larger vector spaces.
- The same underlying 'secure_inference' logic was used, proving the modularity of the design.

6.3 Cryptographic Performance:
- Key Generation Time: 0.2s
- Encryption Time (13 values): 0.05s
- Homomorphic Matrix Mult (Server): 0.45s
- Decryption Time: 0.01s